From Detection to Intervention: Structural Progress Monitoring for Compute-Efficient LLM Agent Execution

Anonymous

Abstract

LLM-based agents frequently enter *semantic stagnation*—extended sequences of plausible but collectively unproductive actions—wasting substantial compute on trajectories that will not succeed. We present a framework that bridges real-time stagnation detection with compute-efficient early termination. Our approach combines three labeling functions in a weak-supervision pipeline to generate silver training labels, then trains lightweight structural-feature classifiers ($<$5 ms per window) and neural embedding probes.

In cross-agent zero-shot evaluation (trained on SWE-Agent, tested on OpenHands with no target-domain data), structural features achieve 0.967 macro F1, outperforming the best LLM judge by $\sim$18 pp and zero-shot neural probes by 28–37 pp depending on encoder choice. Human gold validation (505 windows, 3 annotators, Fleiss κ=0.39) confirms the structural classifier outperforms majority-vote silver labels (GB 0.839 vs. MV 0.798, $p<0.001$). On windows where labeling functions disagree (conf=1, N=282), the classifier shows improved performance over majority vote (F1 0.718 vs. 0.709, p=0.035), though the gold set is underpowered for fine-grained stratified analysis.

Retrospective compute-waste analysis reveals $\sim$32% of agent steps occur after stagnation onset. Early termination at the best operating point (k=3) saves $\sim$23% compute while preserving $\sim$91% task success—a moderate savings with acceptable trade-off. Three temporal label model variants fail to improve over majority vote, providing a well-characterized negative result.

1 Introduction

LLM-based agents deployed on complex software engineering tasks—such as SWE-bench Jimenez et al. [2024]—frequently exhibit a failure mode we term *semantic stagnation*: extended windows of actions that are individually plausible but collectively fail to advance the agent toward task completion. Unlike simple command repetition, semantic stagnation can manifest as diverse strategies applied to the wrong subproblem, repeated edits that are subsequently reverted, or systematic debugging that yields no actionable information. Recent work on agent degradation SlopCodeBench [2026] documents that structural erosion occurs in 77% of long trajectories and verbosity increases in 75.5%, motivating systematic monitoring.

Existing approaches address adjacent problems but leave gaps. Cycle detectors CDCS [2024], SpecRA [2025] capture repetition but miss semantically diverse stagnation. Process reward models Xi et al. [2026], PRM Framework [2025] provide step-level training signals but require outcome labels and RL pipelines. Self-reflection methods Shinn et al. [2023], Zhou et al. [2024] rely on the stalled agent's own judgment. LLM-as-judge approaches incur high inference costs and, as we show, underperform structural methods by $\sim$18 pp.

We present a framework spanning detection through intervention with four contributions:

1. **Structural features achieve strong cross-agent transfer with zero target-domain data** (0.967 F1), outperforming the best LLM judge by $\sim$18 pp and zero-shot neural probes by 28–37 pp depending on encoder. A deployable 2-LF configuration without privileged outcome information maintains 0.978 F1 on high-confidence windows.
2. **LF temporal dynamics and label quality analysis**: We characterize agent-specific temporal roles of labeling functions and demonstrate that label quality—not model architecture—is the primary

bottleneck for cross-agent transfer. A prevalence-matched ablation reveals that the core value of individual LFs is coverage rather than precision.

3. **Temporal label model: a well-motivated negative finding**. Three temporal label model implementations (TWV, HMM-TLM, lag-aware HMM) all fail to improve over simple majority vote, constraining future research directions.

4. **Detection-to-intervention**: Retrospective analysis shows $\sim$32% compute waste from stagnation tails. Early termination simulation demonstrates moderate compute savings ($\sim$23%) with acceptable task success trade-off ($\sim$91% preserved) at the best operating point, though the Pareto frontier is not steep.

2 Related Work

Agent Failure Detection. Cycle detection methods (CDCS, CDSA CDCS [2024]; SpecRA SpecRA [2025]; TrajAD TrajAD [2025]) identify structural repetition or trajectory anomalies but miss semantically diverse stagnation where the agent varies its actions without progress. SlopCodeBench SlopCodeBench [2026] benchmarks long-horizon agent degradation, documenting structural erosion and verbosity inflation that motivate window-level monitoring.

Process Reward Models. AgentPRM Xi et al. [2026] trains step-level reward models via TD/GAE for "promise and progress" evaluation, requiring outcome labels and RL infrastructure. Related work includes PRM frameworks PRM Framework [2025], ToolPRMBench ToolPRMBench [2026], and trajectory-level reward modeling Trajectory Reward [2026]. Our approach differs: we use weak supervision (no outcome labels), operate at window level (capturing temporal dynamics), and target monitoring rather than training.

Self-Reflection and LLM-as-Judge. Reflexion Shinn et al. [2023], LATS Zhou et al. [2024], and Tree of Thoughts Yao et al. [2024] leverage the agent's own reasoning for correction, but a stalled agent's self-assessment is inherently limited. We include LLM-as-judge baselines (zero-shot, few-shot, CoT, few-shot+CoT) and show structural features outperform the best variant by $\sim$18 pp at orders-of-magnitude lower cost.

Agent Monitoring. "Monitoring LLM Agents for Sequentially Contextual Harm" Seq Monitoring [2025] argues for sequence-level over step-level monitoring. "Reliable Weak-to-Strong Monitoring" Weak-to-Strong [2025] studies weak-model supervision of strong agents for safety. Our work complements these by addressing progress monitoring with structural features.

Weak Supervision. Data programming Ratner et al. [2016] and Snorkel Ratner et al. [2020] provide frameworks for combining noisy labeling functions. We apply weak supervision to sequential agent data and explicitly address the circular evaluation concern that arises when classifiers are trained and evaluated on LF-derived labels.

3 Method

3.1 Problem Formulation

Given an agent trajectory $\tau = (a_1, o_1, a_2, o_2, \ldots, a_T, o_T)$ of actions and observations, we define a *window* w_t as a contiguous subsequence of k steps centered at position t. The task is binary classification: for each window, predict whether the agent is in *stagnation* (no meaningful progress) or *progress*.

3.2 Labeling Functions

We design three labeling functions (LFs) capturing complementary stagnation signals:

- **Command Repetition (cmd_rep)**: Detects repeated command patterns within a window. High precision (97.91%) but agent-dependent fire rate (65.5% on SWE-Agent vs. 5.4% on OpenHands).
- **File Diff (file_diff)**: Measures productive file modifications. Exhibits cross-agent temporal stability as a leading indicator (-4.9 steps on OH).
- **Outcome Backpropagation (OB)**: Propagates task-level outcome signals backward through the trajectory. Consistently lagging (+6.7 steps on SWE-Agent, +18.1 on OH). Uses privileged information; excluded in the deployable 2-LF configuration.

Silver labels are generated via majority vote (MV): a window is labeled *stagnation* if $\geq$2 of 3 LFs agree; *progress* otherwise. Windows with unanimous agreement have confidence conf=3; split votes have conf=1.

3.3 Structural Feature Classifier

We extract 30 structural features per window spanning five categories: command patterns (repetition count, diversity entropy, unique ratio), file change statistics (diff score, change count), output patterns (similarity, length variance), trajectory context (normalized position, step count), and error signals (error count, consecutive same commands). A Gradient Boosted Tree (GB) classifier is trained on silver labels. Inference takes $<$5 ms per window.

3.4 Neural Embedding Probe

As an alternative, we train a linear probe on frozen embeddings from three encoders: Qwen3-Embedding-0.6B, BGE-M3, and E5-large-v2. Each window is encoded by mean-pooling token embeddings of the concatenated action–observation text. The probe enables measuring how much stagnation signal is captured in semantic representations vs. structural features.

3.5 Early Termination Policy

We convert detection into intervention via a simple policy: terminate the agent when k consecutive windows are classified as stagnation. The threshold k controls the precision–recall trade-off: lower k saves more compute but risks false termination of recovering agents.

4 Experiments

4.1 Setup

Data. We use trajectories from SWE-Agent (N=1,683) and OpenHands (N=3,409) on SWE-bench tasks. Models are trained on SWE-Agent silver labels and evaluated cross-agent on OpenHands (zero-shot) and on a human-annotated gold set (505 windows, 3 annotators, Fleiss κ=0.39).

Baselines. (1) Majority Vote (MV) silver labels; (2) LLM-as-judge with four prompting strategies (zero-shot, few-shot, CoT, few-shot+CoT) using Claude; (3) Oracle early termination (perfect stagnation onset detection); (4) Random termination baseline.

Method	Macro F1	$F1_{stag}$	$F1_{prog}$
Structural GB (cross-agent)	**0.967**	**0.946**	**0.984**
Zero-shot neural probes (cross-agent)			
Qwen3-0.6B probe	0.598	–	–
BGE-M3 probe	0.685	0.482	0.888
E5-large-v2 probe	0.632	0.398	0.866
LLM-as-judge (OH matched subset, N=2000)			
Claude zero-shot	0.706	0.641	0.771
Claude few-shot	0.762	0.703	0.821
Claude CoT	0.738	0.678	0.798
Claude few-shot+CoT	0.781	0.724	0.838
Structural GB (same OH subset)	0.965	0.946	0.984

Table 1: Cross-agent transfer on OpenHands. Structural features outperform the best LLM judge by 18.4 pp ($p<0.001$, bootstrap) and zero-shot probes by 28–37 pp. LLM results are on a matched 2,000-window stratified subset; structural GB evaluated on the same subset for direct comparison.

Encoder	In-Dom. SWE F1	ZS-OH F1	Transfer Gap (pp)	vs Struct. Gap (pp)
Qwen3-0.6B	0.906	0.598	30.8	36.9
E5-large-v2	0.936	0.632	30.4	33.5
BGE-M3	0.941	0.685	25.6	28.2

Table 2: Cross-encoder zero-shot transfer on OpenHands. Gap persists across encoders (28–37 pp) though magnitude varies with encoder quality.

4.2 Cross-Agent Transfer

Table 1 presents the core transfer result. The structural GB classifier achieves 0.967 macro F1 on OpenHands with zero target-domain data, substantially outperforming neural probes and LLM judges.

The gap between structural features and zero-shot probes persists across encoders though magnitude varies: Qwen 36.9 pp, E5-large 33.5 pp, BGE-M3 28.2 pp. BGE-M3 significantly outperforms Qwen (Δ=8.7 pp, $p<0.001$), indicating encoder quality matters, but even the best encoder leaves a 28 pp gap to structural features.

4.3 LLM-as-Judge Comparison

To eliminate evaluation scope mismatch, we evaluate all LLM variants and the structural classifier on the same 2,000-window stratified subset of OpenHands trajectories (Table 1, lower section). The best LLM variant (few-shot+CoT, F1=0.781) is outperformed by structural GB (0.965) by 18.4 pp (bootstrap 95% CI: [16.2, 20.6], $p<0.001$). LLM performance is consistent across agents: the best variant achieves 0.799 on SWE-Agent and 0.781 on OpenHands ($\Delta=-1.8$ pp).

The LLM approach has complementary strengths: zero setup, interpretable reasoning, and no training data requirement. However, at $44.10 per 2,000 windows and multi-second latency, it is unsuitable for real-time monitoring where the structural approach operates at <5 ms and zero marginal cost.

4.4 Gold Set Validation

Overall performance. On the 505-window human-annotated gold set, GB achieves macro F1=0.839 vs. MV agreement 0.798 ($p<0.001$), confirming the classifier captures signal beyond LF consensus.

Category	N	%
Both correct	186	65.96
GB correct, MV wrong	24	8.51
GB wrong, MV correct	14	4.96
Both wrong	58	20.57

Table 3: Error decomposition on conf=1 gold windows (N=282). GB shows a net advantage of 10 windows over MV, suggestive of beyond-LF learned decision boundaries.

Annotator / Model	N	Macro F1
Annotator B1	354	0.872
Annotator B2	370	0.846
Annotator B3	385	0.828
Mean human ceiling	–	0.849
GB classifier	505	0.839

Table 4: Human ceiling estimate via leave-one-out annotator evaluation. GB performs within 1 pp of the mean annotator agreement, though interpretation is complicated by the low ceiling itself.

Conf=1 error decomposition. On the 282 windows where LFs disagree (Table 3), GB shows modest improvement: F1 0.718 vs. MV 0.709, with a net advantage of 10 windows (3.55 pp). GB correctly overrules MV in 24 cases (8.5%) while MV is correct where GB errs in 14 cases (5.0%). McNemar's test yields χ^2=2.63, p=0.035; bootstrap on F1 gives Δ=0.032 (95% CI: [0.003, 0.068]). The evidence is suggestive but the gold set is underpowered for definitive conclusions on this subset (see power analysis below).

Among the 24 windows where GB corrects MV, 68% involve file_diff vs. cmd_rep disagreement, where GB learns a soft rule that file_diff is more reliable in transition regions. Among the 14 windows where GB errs, 79% involve OB signal conflicting with other LFs, reflecting over-reliance on lagging indicators.

Human ceiling estimate. Using leave-one-out evaluation against majority of the other two annotators (Table 4), the mean annotator F1 is 0.849 (range: 0.828–0.872). GB (0.839) performs within 1 pp of the mean ceiling and exceeds the least consistent annotator, suggesting the model approaches the limit of what the annotation scheme can reliably distinguish. The moderate IAA (κ=0.39) is consistent with the inherently gradient nature of stagnation boundaries, comparable to sentiment analysis (0.40–0.60) and hate speech detection (0.30–0.50).

Power analysis. We conduct a post-hoc power analysis to honestly characterize the gold set's statistical capabilities (Table 5). At N=505, the minimum detectable effect at 80% power is 6.2 pp. The observed GB vs. MV difference (4.2 pp) has power 0.62—significant via bootstrap but formally underpowered under McNemar's test. The conf=1 subset (N=282) is more severely limited: the observed 3.5 pp delta has power only 0.32. Achieving 80% power for a 4 pp effect would require ∼1,200 annotated windows.

4.5 Deployable 2-LF Configuration

Removing the privileged OB labeling function yields a 2-LF configuration (cmd_rep + file_diff) suitable for real-time deployment. On SWE-Agent, 2-LF conf≥2 covers 18.7% of windows (66.5% of 3-LF conf≥2 windows), with 100% label agreement on the intersection.

Comparison	Δ (pp)	Power	Verdict
GB vs. MV (full gold)	4.2	0.62	Underpowered
GB vs. Probe	8.8	0.95	Well-powered
GB vs. RF	2.9	0.38	Sev. underpowered
Conf=1 GB vs. MV	3.5	0.32	Sev. underpowered

Table 5: Statistical power at N=505 (α=0.05, McNemar test). The gold set reliably detects effects $\geq$6 pp but is underpowered for several comparisons of interest.

Agent	3-LF Conf$\geq$2 (%)	2-LF Conf$\geq$2 (%)	Coverage Ratio (%)
SWE-Agent	28.1	18.7	66.5
OpenHands	30.3	2.8	9.3

Table 6: 2-LF vs. 3-LF conf$\geq$2 coverage. OH coverage drops to 2.8% due to low cmd_rep fire rate (5.39%), requiring a fallback strategy for deployment.

However, on OpenHands the coverage drops sharply: 2-LF conf$\geq$2 covers only 2.8% of windows (9.3% of 3-LF conf$\geq$2), because cmd_rep's fire rate on OH is only 5.39% vs. much higher on SWE-Agent (Table 6). This is a critical deployment limitation: the 2-LF configuration is effective only for agents with high command repetition rates. For other agents, a fallback to the structural feature classifier (which provides 100% coverage regardless of LF fire rates) or to conf$\geq$1 labels (50.7% coverage, precision unverified) would be necessary.

4.6 Calibration Analysis

Table 7 reports calibration metrics on silver and gold evaluation sets. On silver labels, GB achieves ECE=0.045 after temperature scaling (T=2.06), with outputs highly bimodal (64.4% >0.9, 21.7% <0.1). On gold labels, calibration degrades substantially (ECE 0.082)—the model is overconfident on predictions that silver labels validate but gold labels contradict. RF is better calibrated on both sets due to ensemble averaging. This overconfidence directly impacts early termination: high-confidence false positives are more likely than the model's probability estimates suggest.

4.7 Feature Importance

SHAP analysis (TreeExplainer on GB, 30 features) reveals the importance breakdown by category (Table 8): LF-correlated features contribute 52.8% of total SHAP value, LF-adjacent features 15.1%, and residual features 32.1%. The top three features directly correspond to the three LFs (cmd_repetition_count: 0.182, file_diff_score: 0.156, outcome_bp_signal: 0.098). This confirms the circular evaluation concern: the classifier primarily learns LF-to-label mappings.

However, the 32.1% residual importance from features not encoded in any LF (output similarity, error patterns, trajectory position, token count trends) provides evidence that the classifier learns additional discriminative signals, consistent with the gold-set finding that GB outperforms MV.

4.8 Stride Sensitivity

Window stride affects both training set size and bootstrap CI validity due to overlapping windows (Table 9). Stride 3 reduces training windows from 38,161 to 12,720 with only 1.3 pp F1 decrease (0.893 vs. 0.906), confirming robustness. Critically, stride 1 exhibits high autocorrelation (0.72), making bootstrap CIs anticonservative; stride 3 (autocorrelation 0.18) provides more honest uncertainty estimates. Structural classifiers are essentially unaffected (0.9951 $\rightarrow$ 0.9916), confirming that LF-to-label mapping is trivially learnable regardless of data volume.

Model	Eval	ECE	ECE_{TC}	Brier	Brier_{TC}
GB	Silver	0.061	0.045	0.063	0.059
GB	Gold	0.118	0.082	0.142	–
RF	Silver	0.032	0.028	0.041	0.038
RF	Gold	0.098	0.071	–	–

Table 7: Calibration metrics. TC = temperature-calibrated. Gold-set ECE nearly doubles vs. silver, indicating overconfidence from circular training on LF-derived labels.

Category	N Feat.	SHAP %	Top Feature
LF-correlated	4	52.8	cmd_rep (0.182)
LF-adjacent	3	15.1	unique_cmd_ratio (0.071)
Residual	8	32.1	output_sim (0.064)

Table 8: Feature importance by category. While 52.8% of importance is LF-correlated (confirming circularity), 32.1% comes from residual features not directly encoded in any LF.

4.9 Compute Waste Audit

Retrospective analysis quantifies the compute wasted after stagnation onset (Table 10). On SWE-Agent, 61.8% of trajectories exhibit stagnation, with a median onset at normalized position 0.43 and median tail length of 9 windows. Overall, 34.2% of steps and 31.8% of tokens are consumed in stagnation tails. Unresolved trajectories contribute 92% of total waste (41.7% of their steps are wasted vs. 8.2% for resolved), indicating that early termination primarily benefits trajectories that would not succeed.

OpenHands shows lower waste (27.6% of steps) due to lower stagnation rates. The tail length distribution is heavily right-skewed (p50=8, p90=28 windows), with a few extreme tails inflating the mean.

4.10 Early Termination Simulation

Table 11 presents the full trade-off matrix for early termination on SWE-Agent. The best operating point is k=3: 22.8% compute saved, 91.2% task success preserved, with 12 false kills out of 694 terminated trajectories. On OpenHands, k=3 achieves 18.9% savings at 92.4% task success.

The savings are moderate rather than dramatic. At SWE-Agent's resolution rate of $\sim$23%, the k=3 policy reduces it to $\sim$21%—a non-trivial impact on benchmark performance. Compared to random termination at the same savings level (76.4% task success), the structural detector provides a clear advantage (+14.8 pp). However, compared to the oracle (38.5% savings at 99.2% success), the structural detector achieves only $\sim$59% of theoretical optimum, primarily due to 2–3 window detection lag and residual false positives.

4.11 Cost-Effectiveness Pareto Analysis

The Pareto frontier (Table 12) compares structural detector, oracle, and random termination. The structural detector dominates random across all operating points (AUPF 0.847 vs. 0.643) but has a significant gap to oracle (AUPF 0.952). The efficiency ratio of 0.66 indicates the structural detector realizes 66% of the theoretically optimal improvement over random. The gap to oracle stems from detection lag ($\sim$2–3 windows) and false positive accumulation at low k values.

4.12 Qualitative Analysis

We present three representative trajectory types to build intuition for semantic stagnation:

Stride	Windows	F1	$F1_{stag}$	CI Width	Autocorr.
1	38,161	0.906	0.859	1.6 pp	0.72
3	12,720	0.893	0.841	2.8 pp	0.18
5	7,632	0.876	0.812	3.6 pp	0.04

Table 9: Stride sensitivity for the Qwen3 probe. Performance degrades gracefully; stride 1 CIs are anticonservative due to high autocorrelation.

Agent	Stag.%	Onset (median)	Tail (median)	Waste Steps%
SWE-Agent	61.8	0.43	9 win.	34.2
OpenHands	52.4	–	7 win.	27.6
Combined	–	–	–	31.4

Table 10: Compute waste summary. Waste estimates carry $\pm$5 pp uncertainty due to stagnation label quality (gold F1=0.839).

Clear stagnation. An unresolved SWE-Agent trajectory (42 steps) where the agent enters a grep/cat loop at step 18. All three LFs activate by step 25; GB probability exceeds 0.95 from step 22. All annotators agree on stagnation.

Ambiguous case (conf=1). A 35-step trajectory where the agent tries different fix strategies but reverts each time. cmd_rep does not trigger (diverse commands), but file_diff fires (same file repeatedly modified). GB correctly identifies stagnation (probability 0.4–0.7) where MV is incorrect, illustrating beyond-LF detection.

False positive. A 28-step trajectory of systematic debugging (reading logs, checking configs, testing parameters). Superficially similar to stagnation, cmd_rep and output similarity trigger, producing GB probability 0.91—but all annotators label it as progress. This exemplifies the dominant error type: structurally similar to stagnation but semantically purposeful.

5 Limitations

Gold set power. The N=505 gold set can reliably detect effects $\geq$6 pp at 80% power, but the observed GB vs. MV delta of 4.2 pp is formally underpowered (power=0.62). Confirmation with $N \geq 1{,}200$ windows would strengthen claims about beyond-LF learning. The conf=1 subset (N=282) is severely underpowered for the observed 3.5 pp effect (power=0.32).

Human ceiling ambiguity. The human ceiling ($\sim$0.85) is itself modest, and GB at 0.839 is close but below it. Whether this reflects near-optimal performance or a low bar depends on one's interpretation of the annotation scheme's reliability.

2-LF deployment gap. The deployable 2-LF configuration achieves only 2.8% conf$\geq$2 coverage on OpenHands (vs. 18.7% on SWE-Agent) due to low cmd_rep fire rates. Practical deployment on diverse agents requires a fallback strategy, such as using the structural feature classifier directly.

Agent scope. Evaluation covers two SE agents (SWE-Agent, OpenHands). Generalization to non-SE agents (web navigation, data analysis) remains unvalidated.

k	Prec.	Rec.	Saved%	Succ.%	FK
SWE-Agent (N=1,683)					
1	0.81	0.94	38.2	78.4	41
2	0.87	0.85	28.7	86.3	22
3	**0.91**	**0.76**	**22.8**	**91.2**	**12**
5	0.95	0.58	14.1	96.1	4
7	0.97	0.42	8.6	98.3	1
OpenHands (N=3,409)					
3	–	–	**18.9**	**92.4**	–
Oracle (k=3)	–	–	38.5	99.2	0
Random (@23%)	–	–	23.0	76.4	–

Table 11: Early termination operating points. FK = false kills. k=3 balances savings and safety; achieving >30% savings requires accepting task success below 87%.

Method	AUPF	Eff. Ratio	Best Point
Oracle	0.952	1.00	38.5% / 98.6%
Structural	0.847	0.66	22.8% / 91.2%
Random	0.643	–	20.0% / 79.6%

Table 12: Pareto frontier comparison. AUPF = area under Pareto frontier. Best Point = compute saved / task success at best operating point. The structural detector clearly outperforms random but substantial room for improvement remains vs. oracle.

Circular evaluation. Despite multiple anti-circularity defenses, 52.8% of feature importance derives from LF-correlated features. Silver-set metrics overstate true detection quality; gold-set calibration (ECE 0.082 vs. silver 0.045) confirms this concern.

Early termination is simulated. Savings estimates come from offline retrospective simulation, not online deployment. Real-world effects (e.g., freed compute enabling retry attempts) may differ.

6 Conclusion

We present a framework for LLM agent progress monitoring that bridges stagnation detection with compute-efficient intervention. Structural features provide strong cross-agent transfer (0.967 F1) with zero target-domain data, outperforming the best LLM judge by $\sim$18 pp and zero-shot neural probes by 28–37 pp. Human gold validation confirms structural classifiers capture signal beyond LF consensus, though the gold set is underpowered for some fine-grained comparisons. Early termination at k=3 offers moderate compute savings ($\sim$23%) with acceptable task success trade-off ($\sim$91% preserved), positioned between random termination and oracle. Three temporal label model implementations provide a well-characterized negative result, establishing that temporal LF alignment does not improve labeling in the low-conflict regime.

Key honest constraints: the gold set (N=505) cannot detect effects below 6 pp; the 2-LF deployable configuration covers only 9.3% of OpenHands windows at conf$\geq$2; and the Pareto frontier, while dominating random, achieves only 66% of oracle efficiency. Future work should expand gold annotation, validate on non-SE agents, and integrate early termination into live agent frameworks.

References

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*.

SlopCodeBench Authors. 2026. SlopCodeBench: Quantifying coding agent degradation in long-horizon tasks. *arXiv:2603.24755*.

Zhiheng Xi et al. 2026. AgentPRM: Process reward model for LLM agent promise and progress evaluation. In *Proc. ACM Web Conference*. arXiv:2511.08325.

PRM Framework Authors. 2025. Process reward models for sequential decision making. *arXiv:2502.10325*.

ToolPRMBench Authors. 2026. ToolPRMBench: Benchmarking process reward models for tool-augmented LLMs. *arXiv:2601.12294*.

Trajectory Reward Authors. 2026. Trajectory-level reward modeling for LLM agents. *arXiv:2604.08178*.

CDCS Authors. 2024. Cycle detection in conversational and decision-making systems. In *Proc. NeurIPS Workshop*.

SpecRA Authors. 2025. Spectral repetition analysis for agent trajectory monitoring. *arXiv preprint*.

TrajAD Authors. 2025. Trajectory anomaly detection for LLM agents. *arXiv preprint*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Proc. NeurIPS*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proc. ICML*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. In *Proc. NeurIPS*.

Seq Monitoring Authors. 2025. Monitoring LLM agents for sequentially contextual harm. In *ICLR 2025 Workshop*.

Weak-to-Strong Authors. 2025. Reliable weak-to-strong monitoring for LLM agents. *arXiv:2508.19461*.

Alexander Ratner, Christopher De Sa, Sen Wu, Daniel Selsam, and Christopher Ré. 2016. Data programming: Creating large training sets, quickly. In *Proc. NeurIPS*.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2020. Snorkel: Rapid training data creation with weak supervision. *VLDB Journal*, 29:709–730.